

Emitting messages at compile time

Document #: P2758R4 [\[Latest\]](#) [\[Status\]](#)
Date: 2025-01-07
Project: Programming Language C++
Audience: CWG, LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 General compile-time debugging](#)
 - [2.2 Prior Work](#)
- [3 To `std::format` or not to `std::format`?](#)
- [4 Improving compile-time diagnostics](#)
 - [4.1 Predictability](#)
 - [4.2 Predictability of Errors](#)
 - [4.3 Error-Handling in General](#)
 - [4.4 Errors in Constraints](#)
 - [4.5 Warnings and Tagging](#)
 - [4.5.1 Tag Restrictions](#)
 - [4.5.2 Tagging in other interfaces](#)
 - [4.6 Constexpr-Erroneous Values](#)
 - [4.6.1 Immediate Escalation](#)
 - [4.6.2 Exceptions](#)
- [5 Proposal](#)
 - [5.1 Implementation Experience](#)
- [6 Wording](#)
- [7 References](#)

§ 1 Revision History

For R4: wording. Re-targeting towards CWG and LEWG. Introduced concept of constexpr-erroneous both for proper wording and to handle an escalating issue.

For [\[P2758R3\]](#): Clean-up the paper to account for other papers ([\[P2741R3\]](#) and [\[P2738R1\]](#)) being adopted. More discussion of tags, which are added to every API. Expanding wording.

For [\[P2758R2\]](#): clarify the section about [SFINAE-friendliness](#), reduced the API to just one error function, and adding a [warning API](#) as well.

For [\[P2758R1\]](#): [\[P2758R0\]](#) and [\[P2741R0\]](#) were published at the same time and had a lot of overlap. Since then, [\[P2741R3\]](#) was adopted. As such, this paper no longer needs to propose the same thing. That part of the paper has been removed. This revision now only adds library functions that emit messages at compile time.

§ 2 Introduction

Currently, our ability to provide diagnostics to users is pretty limited. There are two ways that libraries can provide diagnostics to users right now.

First, there is `static_assert`. At the time of writing the initial revision of this paper, `static_assert` was limited to only accepting a string literal. However, since then, [\[P2741R3\]](#) has been adopted for C++26, which allows user-generated messages. That is a fantastic improvement.

The second way is via forced constant evaluation failures. Consider the example:

```
auto f() -> std::string {  
    return std::format("{} {:d}", 5, "not a number");  
}
```

One of the cool things about `std::format` is that the format string is checked at compile time. The above is ill-formed: because `d` is not a valid format specifier for `const char*`. What is the compiler error that you get here?

```
<source>:6): error C7595: 'fmt::v9::basic_format_string<char,int,const char (&)[13]>::basic_format_string': call to
immediate function is not a constant expression
C:\data\libraries\installed\x64-windows\include\fmt\core.h(2839): note: failure was caused by call of undefined
function or one not declared 'constexpr'
C:\data\libraries\installed\x64-windows\include\fmt\core.h(2839): note: see usage of
'fmt::v9::detail::error_handler::on_error'
```

— GCC

```
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format: In function 'std::string f()':
<source>:6:23: in 'constexpr' expansion of 'std::basic_format_string<char, int, const char (&)[13]>("{ {} :d}")'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3634:19: in 'constexpr' expansion of
'__scanner.std::__format::Checking_scanner<char, int, char [13]>::
<anonymous>.std::__format::Scanner<char>::M_scan()'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3448:30: in 'constexpr' expansion of
'((std::__format::Scanner<char>*)this)->std::__format::Scanner<char>::M_on_replacement_field()'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3500:15: in 'constexpr' expansion of
'((std::__format::Scanner<char>*)this)->std::__format::Scanner<char>::M_format_arg(__id)'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3572:33: in 'constexpr' expansion of
'((std::__format::Checking_scanner<char, int, char [13]>*)this)->std::__format::Checking_scanner<char, int,
char [13]>::M_parse_format_spec<int, char [13]>(__id)'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3589:36: in 'constexpr' expansion of
'((std::__format::Checking_scanner<char, int, char [13]>*)this)->std::__format::Checking_scanner<char, int,
char [13]>::M_parse_format_spec<char [13]>((__id - 1))'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:3586:40: in 'constexpr' expansion of
'__f.std::formatter<char [13], char>::parse((std::__format::Checking_scanner<char, int, char [13]>*)this)-
>std::__format::Checking_scanner<char, int, char [13]>::<anonymous>.std::__format::Scanner<char>::M_pc)'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:1859:26: in 'constexpr' expansion of
'((std::formatter<char [13], char>*)this)->std::formatter<char [13],
char>::M_f.std::__format::formatter_str<char>::parse((* &__pc))'
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:823:48: error: call to non-'constexpr' function
'void std::__format::__failed_to_parse_format_spec()'
823 | __format::__failed_to_parse_format_spec();
| ~~~~~^~~~~~
/opt/compiler-explorer/gcc-trunk-20230108/include/c++/13.0.0/format:185:3: note: 'void
std::__format::__failed_to_parse_format_spec()' declared here
185 | __failed_to_parse_format_spec()
| ^~~~~~
```

— Clang (with libstdc++, libc++ doesn't implement <format> yet):

```
<source>:6:24: error: call to consteval function 'std::basic_format_string<char, int, const char (&)[13]>::basic_format_string<char[8]>' is not a constant expression
return std::format("{} {} :d", 5, "not a number");
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:823:2: note:
non-constexpr function '__failed_to_parse_format_spec' cannot be used in a constant expression
__format::__failed_to_parse_format_spec();
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:1859:21:
note: in call to '&__f.M_f->parse(__scanner._Scanner::M_pc)'
{ return __f.parse(__pc); }
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:3586:35:
note: in call to '&__f->parse(__scanner._Scanner::M_pc)'
this->M_pc.advance_to(__f.parse(this->M_pc));
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:3589:6:
note: in call to '&__scanner->M_parse_format_spec(0)'
__M_parse_format_spec<Tail...>(__id - 1);
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:3572:3:
note: in call to '&__scanner->M_parse_format_spec(1)'
__M_parse_format_spec<Args...>(__id);
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:3500:2:
note: in call to '&__scanner->M_format_arg(1)'
__M_format_arg(__id);
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:3448:7:
note: in call to '&__scanner->M_on_replacement_field()'
__M_on_replacement_field();
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:3634:12:
note: in call to '&__scanner->M_scan()'
__scanner.M_scan();
^
<source>:6:24: note: in call to 'basic_format_string("{} {} :d")'
return std::format("{} {} :d", 5, "not a number");
^
/opt/compiler-explorer/gcc-snapshot/lib/gcc/x86_64-linux-gnu/13.0.0/../../../../include/c++/13.0.0/format:185:3: note:
declared here
__failed_to_parse_format_spec()
^
```

— GCC, using {fmt} trunk instead of libstdc++:

```

/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h: In function 'std::string f()':
<source>:6:23: in 'constexpr' expansion of 'fmt::v9::basic_format_string<char, int, const char (&)[13]>("{ } {d}")'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2847:40: in 'constexpr' expansion of
    'fmt::v9::detail::parse_format_string<true, char, format_string_checker<char, int, char [13]> >
    (((fmt::v9::basic_format_string<char, int, const char (&)[13]>*)this)->fmt::v9::basic_format_string<char, int,
    const char (&)[13]>::str_, fmt::v9::detail::format_string_checker<char, int, char [13]>
    (fmt::v9::basic_string_view<char>(((const char*)s))))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2583:44: in 'constexpr' expansion of
    'fmt::v9::detail::parse_replacement_field<char, format_string_checker<char, int, char [13]>&>((p + -1), end,
    (* & handler))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2558:38: in 'constexpr' expansion of '(& handler)-
    >fmt::v9::detail::format_string_checker<char, int, char [13]>&>(const char*, const char*, format_string_checker<char, int,
    char [13]>&>((const char*)s))->id_adapter::arg_id,
    (begin + 1), end)'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2727:51: in 'constexpr' expansion of
    '(((fmt::v9::detail::format_string_checker<char, int, char [13]>*)this)-
    >fmt::v9::detail::format_string_checker<char, int, char [13]>::parse_funcs_[id]
    (((fmt::v9::detail::format_string_checker<char, int, char [13]>*)this)-
    >fmt::v9::detail::format_string_checker<char, int, char [13]>::context_))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2641:17: in 'constexpr' expansion of
    'f.fmt::v9::formatter<const char*, char, void>::parse<fmt::v9::detail::compile_parse_context<char> >((* &
    ctx))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2784:35: in 'constexpr' expansion of
    'fmt::v9::detail::parse_format_specs<char>((& ctx)->fmt::v9::detail::compile_parse_context<char>::
    <anonymous>.fmt::v9::basic_format_parse_context<char>::begin(), (& ctx)-
    >fmt::v9::detail::compile_parse_context<char>::<anonymous>.fmt::v9::basic_format_parse_context<char>::end(),
    ((fmt::v9::formatter<const char*, char, void>*)this)->fmt::v9::formatter<const char*, char, void>::specs_,
    ctx.fmt::v9::detail::compile_parse_context<char>::<anonymous>, type)'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2468:37: in 'constexpr' expansion of
    'parse_presentation_type.fmt::v9::detail::parse_format_specs<char>(const char*, const char*,
    dynamic_format_specs<&, fmt::v9::basic_format_parse_context<char>&, type)::unnamed struct::operator()
    (fmt::v9::presentation_type::dec, ((int)integral_set))'
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:2395:49: error: call to non-'constexpr' function 'void
    fmt::v9::detail::throw_format_error(const char*)'
2395 |         if (!in(arg_type, set)) throw_format_error("invalid format specifier");
    |                                     ^~~~~~
/opt/compiler-explorer/libs/fmt/trunk/include/fmt/core.h:646:27: note: 'void fmt::v9::detail::throw_format_error(const
    char*)' declared here
646 | FMT_NORETURN FMT_API void throw_format_error(const char* message);
    |                                     ^~~~~~

```

All the compilers reject the code, which is good. MSVC gives you no information at all. Clang indicates that there's something wrong with some format spec, but doesn't show enough information to know what types are involved (is it the `5` or the `"not a number"`?). GCC does the best in that you can actually tell that the problem argument is a `char[13]` (if you really carefully peruse the compile error), but otherwise all you know is that there's *something* wrong with the format spec.

This isn't a standard library implementation problem - the error gcc gives when using `{fmt}` isn't any better. If you carefully browse the message, you can see that it's the `const char*` specifier that's the problem, but otherwise all you know is that it's invalid.

The problem here is that the only way to "fail" here is to do something that isn't valid during constant evaluation time, like throw an exception or invoke an undefined function. And there's only so much information you can provide that way. You can't provide the format string, you can't point to the offending character.

Imagine how much easier this would be for the end-user to determine the problem and then fix if the compiler error you got was something like this:

```

format("{ } {d}", int, const char*)
      ^      ^
'd' is an invalid type specifier for arguments of type 'const char*'

```

That message might not be perfect, but it's overwhelmingly better than anything that's possible today. So we should at least make it possible tomorrow.

§ 2.1 General compile-time debugging

The above two sections were about the desire to emit a compile error, with a rich diagnostic message. But sometimes we don't want to emit an *error*, we just want to emit *some information*.

When it comes to runtime programming, there are several mechanisms we have for debugging code. For instance, you could use a debugger to step through it or you could litter your code with print statements. When it comes to *compile-time* programmer, neither option is available. But it would be incredibly useful to be able to litter our code with *compile-time* print statements. This was the initial selling point of Circle: want compile-time prints? That's just `@meta printf`.

There's simply no way I'm aware of today to emit messages at compile-time other than forcing a compile error, and even those (as hinted at above) are highly limited.

§ 2.2 Prior Work

[N4433] previously proposed extending `static_assert` to support arbitrary constant expressions. That paper was discussed in [Lenexa in 2015](#). The minutes indicate that there was concern about simply being able to implement a useful format in `constexpr` (`{fmt}` was just v1.1.0 at the time). Nevertheless, the paper was well received, with a vote of 12-3-9-1-0 to continue work on the proposal. Today, we know we can implement a useful format in `constexpr`. We already have it!

[P0596R1] previously proposed adding `std::constexpr_trace` and `std::constexpr_assert` facilities - the former as a useful compile-time print and the latter as a useful compile-time assertion to emit a useful message. That paper was discussed in [Belfast in 2019](#), where these two facilities were very popular (16-8-1-0-0 for compile-time print and 6-14-2-0-0 for compile-time assertion). The rest of the discussion was about broader compilation models that isn't strictly related to these two.

In short, the kind of facility I'm reviving here were already previously discussed and received *extremely favorably*. 15-1, 24-0, and 20-0. It's just that then the papers disappeared, so I'm bringing them back.

§ 3 To `std::format` or not to `std::format`?

That is the question. Basically, when it comes to emitting some kind of text (via whichever mechanism - whether `static_assert` or a compile-time print or a compile-time error), we have to decide whether or not to bake `std::format` into the API. The advantage of doing so would be ergonomics, the disadvantage would be that it's a complex library to potential bake into the language - and some people might want these facilities in a context where they're not using `std::format`, for whatever reason.

But there's also a bigger issue: while I said above that we have a useful format in `constexpr`, that wasn't *entirely* accurate. The parsing logic is completely `constexpr` (to great effect), but the formatting logic currently is not. Neither `std::format` nor `fmt::format` are declared `constexpr` today. In order to be able to even consider the question of using `std::format` for generating compile-time strings, we have to first ask to what extent this is even feasible.

Initially (as of R0 of this paper), I think there were currently two limitations (excluding just adding `constexpr` everywhere and possibly dealing with some algorithms that happen to not be `constexpr`-friendly):

1. formatting floating-point types is not possible right now (we made the integral part of `std::to_chars()` `constexpr` [P2291R3], but not the floating point).
2. `fmt::format` and `std::format` rely on type erasing user-defined types, which was not possible to do at compile time due to needing to cast back from `void*`.

I am not in a position to say how hard the first of the two is (it's probably pretty hard?), but the second has already been resolved with the adoption of [P2738R1] (and already implemented in at least gcc and clang). That's probably not too much work to get the rest of format working - even if we ignore floating point entirely. Without compile-time type erasure, it's still possible to write just a completely different consteval formatting API - but I doubt people would be too happy about having to redo all that work.

We will eventually have `constexpr std::format`, I'm just hoping that we can do so with as little overhead on the library implementation itself (in terms of lines of code) as possible.

§ 4 Improving compile-time diagnostics

While in `static_assert`, I'm not sure that we can adopt a `std::format()`-based API ¹, for compile-time diagnostics, I think we should. In particular, the user-facing API should probably be something like this:

```
namespace std {
    template<class... Args>
        constexpr void constexpr_print(format_string<Args...> fmt, Args&&... args);
    template<class... Args>
        constexpr void constexpr_warn(format_string<Args...> fmt, Args&&... args);
    template<class... Args>
        constexpr void constexpr_error(format_string<Args...> fmt, Args&&... args);
}
```

But we'll probably still need a lower-level API as well. Something these facilities can be implemented on top of, that we might want to expose to users anyway in case they want to use something other than `std::format` for their formatting needs. Perhaps something like this:

```
namespace std {
    constexpr void constexpr_print_str(string_view);
    constexpr void constexpr_warn_str(string_view);
    constexpr void constexpr_error_str(string_view);
}
```

That is really the minimum necessary, and the nice format APIs can then trivially be implemented by invoking `std::format` and then passing in the resulting `std::string`.

But in order to talk about what these APIs actually do and what their effects are, we need to talk about a fairly complex concept: predictability.

§ 4.1 Predictability

[P0596R1] talks about predictability introducing this example:

```
template<typename> constexpr int g() {
    std::__report_constexpr_value("in g()\n");
    return 42;
}

template<typename T> int f(T(*)[g<T>()]); // (1)
template<typename T> int f(T*);           // (2)
int r = f<void>(nullptr);
```

When the compiler resolves the call to `f` in this example, it substitutes `void` for `T` in both declarations (1) and (2). However, for declaration (1), it is unspecified whether `g<void>()` will be invoked: The compiler may decide to abandon the substitution as soon as it sees an attempt to create “an array of void” (in which case the call to `g<void>` is not evaluated), or it may decide to finish parsing the array declarator and evaluate the call to `g<void>` as part of that.

We can think of a few realistic ways to address/mitigate this issue:

1. Make attempts to trigger side-effects in expressions that are “tentatively evaluated” (such as the ones happening during deduction) ill-formed with no diagnostic required (because we cannot really require compilers to re-architect their deduction system to ensure that the side-effect trigger is reached).
2. Make attempts to trigger side-effects in expressions that are “tentatively evaluated” cause the expression to be non-constant. With our example that would mean that even a call `f<int>(nullptr)` would find (1) to be nonviable because `g<int>()` doesn’t produce a constant in that context.
3. Introduce a new special function to let the programmer control whether the side effect takes place anyway. E.g., `std::is_tentatively_constant_evaluated()`. The specification work for this is probably nontrivial and it would leave it unspecified whether the call to `g<void>` is evaluated in our example.

We propose to follow option 2. Option 3 remains a possible evolution path in that case, but we prefer to avoid the resulting subtleties if we can get away with it.

As well as:

There is another form of “tentative evaluation” that is worth noting. Consider:

```
constexpr int g() {
    std::__report_constexpr_value("in g()\n");
    return 41;
}
int i = 1;
constexpr int h(int p) {
    return p == 0 ? i : 1;
}
int r = g()+h(0); // Not manifestly constant-evaluated but
                  // g() is typically tentatively evaluated.
int s = g()+1;    // To be discussed.
```

Here `g()+h(0)` is not a constant expression because `i` cannot be evaluated at compile time. However, the compiler performs a “trial evaluation” of that expression to discover that. In order to comply with the specification that `__report_constexpr_value` only produce the side effect if invoked as part of a “manifestly constant-evaluated expression”, two implementation strategies are natural:

1. “Buffer” the side effects during the trial evaluation and “commit” them only if that evaluation succeeds.
2. Disable the side effects during the trial evaluation and repeat the evaluation with side effects enabled if the trial evaluation succeeds.

The second option is only viable because “output” as a side effect cannot be observed by the trial evaluation. However, further on we will consider another class of side effects that can be observed within the same evaluation that triggers them, and thus we do not consider option 2 a viable general implementation strategy.

The first option is more generally applicable, but it may impose a significant toll on performance if the amount of side effects that have to be “buffered” for a later “commit” is significant.

An alternative, therefore, might be to also consider the context of a non-constexpr variable initialization to be “tentatively evaluated” and deem side-effects to be non-constant in that case (i.e., the same as proposed for evaluations during deduction). In the example above, that means that `g()+1` would not be a constant expression either (due to the potential side effect by `__report_constexpr_value` in an initializer that is allowed to be non-constant) and thus `s` would not be statically initialized.

Now, my guiding principle here is that if we take some code that currently works and does some constant evaluation, and add to that code a `constexpr_print` statement, the *only* change in behavior should be the addition of output during compile time. For instance:

```
constexpr auto f(int i) -> int {
    std::constexpr_print("Called f({})\n", i);
    return i;
}

int x = f(2);
```

Without the `constexpr_print`, this variable is constant-initialized. With it, it should be also. It would be easier to deal with the language if we didn't have all of these weird rules. For instance, if you want constant initialize, use `constexpr_init`, if you don't, there's no tentative evaluation. But we can't change that, so this is the language we have.

I think buffer-then-commit is right approach. But also for the first example, that tentative evaluation in a manifestly constant evaluated context is *still* manifestly constant evaluated. It's just unspecified whether the call happens. That is: in the first example, the call `f<void>(nullptr)` may or may not print `"in g()\n"`. It's unspecified. It may make `constexpr` output not completely portable, but I don't think any of the alternatives are palatable.

§ 4.2 Predictability of Errors

An interesting follow-on is what happens here:

```
constexpr int data[3] = {1, 4, 9};

constexpr auto f(int i) -> int {
    if (i < 0) {
        std::constexpr_error_str("cannot invoke f with a negative number");
    }
    return data[i];
}

constexpr int a = f(-1);
int b = f(-1);
```

Basically the question is: what are the actual semantics of `constexpr_error`?

If we just say that evaluation (if manifestly constant-evaluated) causes the evaluation to not be a constant, then `a` is ill-formed but `b` would be (dynamically) initialized with `-1`.

That seems undesirable: this is, after all, an error that we have the opportunity to catch. This is the only such case: all other manifestly constant evaluated contexts don't have this kind of fall-back to runtime. So I think it's not enough to say that constant evaluation fails, but rather that the entire program is ill-formed in this circumstance: both `a` and `b` are ill-formed.

We also have to consider the predictability question for error-handling. Here's that same example again:

```
template<typename> constexpr int g(int i) {
    if (i < 0) {
        std::constexpr_error_str("can't call g with a negative number");
    }
    return 42;
}

template<typename T> int f(T(*)[g<T>(-1)]); // (1)
template<typename T> int f(T*);             // (2)
int r = f<void>(nullptr);
```

If `g<T>(-1)` is called, then it'll hit the `constexpr_error_str` call. But it might not be called. I think saying that if it's called, then the program is ill-formed, is probably fine. If necessary, we can further tighten the rules for substitution and actually specify one way or another (actually specify that `g` is *not* invoked because by the time we lexically get there we know that this whole type is ill-formed, or specify that `g` is invoked because we atomically substitute one type at a time), but it's probably not worth the effort.

Additionally, we could take a leaf out of the book of speculative evaluation. I think of the tentative evaluation of `g<T>(-1)` in this second example *quite* differently from the tentative *constant* evaluation of `f(-1)` in the first example. `f` is *always* evaluated, it's just that we have this language hack that it ends up potentially being evaluated two different ways. `g` isn't *necessarily* evaluated. So there is room to treat these different. If `g` is tentatively evaluated, then we buffer up our prints and errors - such that if it eventually *is* evaluated (that overload is selected), we then emit all the prints and errors. Otherwise, there is no output. That is, we specify *no* output if the function isn't selected. Because the evaluation model is different here - that `f` is always constant-evaluated initially - I don't think of these as inconsistent decisions.

§ 4.3 Error-Handling in General

Basically, in all contexts, you probably wouldn't want to *just* `std::constexpr_error`. Well, in a `constexpr_eval` function, that's all you'd have to do. But in a `constexpr` function that might be evaluated at runtime, you probably still want to fail.

But the question is, *how* do you want to fail? There are so many different ways of failing

- throw an exception (of which kind?)
- return some kind of error object (return code, unexpected, `false`, etc.)
- `std::abort()`
- `std::terminate()`
- etc.

Which fallback depends entirely on the circumstance. For `formatter<T>::parse`, one of my motivating examples here, we have to throw a `std::format_error` in this situation. The right pattern there would probably be:

```
if constexpr {
    std::constexpr_error("Bad specifier {}", *it);
} else {
    throw std::format_error(std::format("Bad specifier {}", *it));
}
```

Which can be easily handled in its own API:

```
template <typename... Args>
constexpr void format_parse_failure(format_string<Args...> fmt, Args&&... args) {
    if constexpr {
        constexpr_error(fmt, args...);
    } else {
        throw format_error(format(fmt, args...));
    }
}
```

So we should probably provide that as well (under whichever name).

But that's a format-specific solution. But a similar pattern works just fine for other error handling mechanisms, except for wanting to return an object (unless your return object happens to have a string part - since the two cases end up being very different). I think that's okay though - at least we have the utility.

§ 4.4 Errors in Constraints

Let's take a look again at the example I showed [earlier](#):

```
constexpr auto f(int i) -> int {
    if (i < 0) {
        std::constexpr_error_str("cannot invoke f with a negative number");
    }
    return i;
}

template <int I> requires (f(I) % 2 == 0)
auto g() -> void;
```

Here, `g<2>()` is obviously fine and `g<3>()` will not satisfy the constraints as usual, nothing interesting to say about either call. But what about if we try `g<-1>()`? Based on our currently language rules and what's being proposed here, `f(-1)` is not a constant expression, and the rule we have in 13.5.2.3 [\[temp.constr.atomic\]/3](#) is:

If substitution results in an invalid type or expression, the constraint is not satisfied. Otherwise, the lvalue-to-rvalue conversion is performed if necessary, and E shall be a constant expression of type `bool`.

That is, `g<-1>()` is ill-formed, with our current rules. That would be the consistent choice.

If we want an error to bubble up such that `g<-1>()` would be SFINAE-friendly, that seems like an entirely different construct than `std::constexpr_error_str`: that would be an exception - that the condition could catch and swallow.

§ 4.5 Warnings and Tagging

Consider the call:

```
std::format("x={ } and y=", x, y);
```

The user probably intended to format both `x` and `y`, but actually forgot to write the `{ }` for the second argument. Which means that this call has an extra argument that is not used by any of the formatters. This is, surprisingly to many people, not an error. This is by design - to handle use-cases like

translation, where some of the arguments may not be used, which is an important use-case of `format`. (Note that the opposite case, not providing enough arguments, is a compile error).

However, it is not a use-case that exists in every domain. For many users of `format`, the above (not consuming every `format` argument) is a bug.

One approach that we could take is to allow the `format` library to flag potential misuses in a way that users can opt in to or opt out of. We even have a tool for that already: warnings! If the `format` library could issue a custom diagnostic, like:

```
std::constexpr_warning(  
    "format-too-many-args",  
    "Format string consumed {} arguments but {} were provided.",  
    current_arg, total);
```

Then the implementation could let users opt in with `-Wformat-too-many-args` (or maybe opt out with `-Wno-format-too-many-args`, or maybe some other invocation).

Moreover, even if *some* parts of your application do translation, many others might not. Perhaps rather than globally adding `-Wno-format-too-many-args`, an implementation would allow a `#pragma` to enable (or disable) this particular warning for the duration of a translation unit. Implementations already do this sort of thing, which is exactly what we want. All we need to do is allow a library author to provide a tag.

There are probably many such examples in many libraries. Giving library authors the power to warn users (and users the power to choose their warning granularity) seems very useful.

§ 4.5.1 Tag Restrictions

During an [SG-16 telecon](#), there was some discussion on what the requirements are of the tag we want to pass to `std::constexpr_warning`. For instance, should this be a core language facility so that we can require a string literal?

Unfortunately, I don't think we can require a string literal - since that would prohibit future evolution to add the `format` API on top of `std::constexpr_warning_str` and friends. Such an API would need to forward its argument down to the hypothetical core language feature, at which point we lose "string-literal-ness." We should, however, strongly encourage users to only use string literal tags.

But we do have to have requirements on the tag, since this is going to be something that we want to expose externally as described above - whether as a command-line flag or `#pragma`. So no quotes, semicolons, or other characters with special meaning in command line shells.

My opening bid is that (and I am obviously not a text guy): a tag is only allowed to contain: A-Z, a-z, 0-9, `_`, and `-`. That's a pretty limited set, but it's probably sufficient for the use-case and should not cause problems on shells, etc.

§ 4.5.2 Tagging in other interfaces

[\[P2758R2\]](#) only introduced a tag parameter for warning but not for print or error. SG-16 suggested that each of the interfaces should also accept a tag that could be used to either suppress diagnostics or elevate to an error. This revision adds those parameters as well (for print, optionally, for warning and error, mandatory).

§ 4.6 Constexpr-Erroneous Values

One of the things that came up during Core review is that we need a way to specify how exactly `std::constexpr_error_str` induces failure. The suggestion that Jason Merrill made was that a call to `std::constexpr_error_str` would produce a *constexpr-erroneous value*. Doing so makes the entire expression *constexpr-erroneous*.

We can use that idea to address the example of static initialization [earlier](#). Right now, for static storage duration variables, we try to perform constant initialization. If that succeeds, great. If it doesn't, we fallback to performing dynamic initialization (at runtime). But not all constant initialization failures are the same. Some are simply because initialization could not be done (e.g. calling a non-`constexpr` function, attempting to read some non-`constexpr` variable, etc.) but some are actual bugs that were caught at compile time (e.g. a call to `std::constexpr_error_str`).

We can say that if constant-initialization fails because the initialization was `constexpr-erroneous`, then the program is ill-formed. That would let us make sure that we can catch errors at compile-time instead of unintentionally turning them into runtime failures.

There are two other interesting things to bring up on this topic: escalation and exceptions.

§ 4.6.1 Immediate Escalation

In [\[P2564R3\]](#), we introduced the notion of immediate escalation. That is, a call to a `constexpr` function that isn't constant might lead to the function it is in getting itself turned into a `constexpr` function. This is an important fix to ensure that it is actually possible to run a wide variety of code at compile time.

But the rule is overly broad right now. Consider this reduction from a bug that Jonathan Wakely and I happened to be discussing while I was working on this paper:

```
#include <print>
```



```

template <typename... Args>
void echo(Args&&... args) {
    #ifdef LAMBDA
        [&]{ std::print("{} ", args...); }();
    #else
        std::print("{} ", args...);
    #endif
}

int main() {
    echo();
}

```

Here, we're basically calling `std::print("{}")`, which is ill-formed because we're providing one replacement field but not arguments to format. The mechanism by which this happens is that `"{}"` is used to initialize an object of type `std::format_string<>`, which has a `constexpr` constructor, but will fail to be a constant expression. So the intent is that this is an error right here.

However.

What this actually means is that the construction of the `std::format_string<>` is immediate-escalating. And that causes outer functions to become `constexpr`, if possible. That's not what we actually want to happen here. `constexpr` propagation solves the problem of widening the bubble of what is being constant-evaluated, so that more things become constant, so that constant evaluation can succeed. The typical example here is reading function parameters — they are not constant expressions, so they cause the function to become `constexpr` so that they don't have to be. But in this case, we didn't fail because our expression was insufficiently constant — we failed because our expression was *wrong*!

The result of immediate-escalation here is that the compiler ends up doing more work to produce more confusing error messages. In this case, with the direct function template we still get a reasonable error:

```

<source>:8:16: error: call to constexpr function 'std::basic_format_string<char>("{}")' is not a constant
      expression
      8 |     std::print("{} ", args...);
        |         ^~~~~
In file included from /opt/compiler-explorer/gcc-trunk-20250106/include/c++/15.0.0/print:43,
      from <source>:1:
<source>:8:16:   in 'constexpr' expansion of 'std::basic_format_string<char>("{}")'
/opt/compiler-explorer/gcc-trunk-20250106/include/c++/15.0.0/format:4377:19:   in 'constexpr' expansion
      'std::format::Checking_scanner<char>::std::format::Scanner<char>::_M_scanner.std::format::Scanner<char>::_M_scanner'
/opt/compiler-explorer/gcc-trunk-20250106/include/c++/15.0.0/format:4032:37:   in 'constexpr' expansion
      '>std::format::Scanner<char>::_M_pc.std::format::Scanner<char>::_M_parse_context::std::basic_format_parse_context'
/opt/compiler-explorer/gcc-trunk-20250106/include/c++/15.0.0/format:278:56: error: call to non-'constexpr'
      function 'void std::format::invalid_arg_id_in_format_string()'
      278 |     format::invalid_arg_id_in_format_string();
          |     ~~~~~^~~~~
/opt/compiler-explorer/gcc-trunk-20250106/include/c++/15.0.0/format:224:3: note: 'void
      std::format::invalid_arg_id_in_format_string()' declared here
      224 |     invalid_arg_id_in_format_string()
          |     ^~~~~~

```

But wrap it in a lambda and the error ceases to make sense to most people:

```

<source>:6:38: error: call to constexpr function '<lambda closure object>echo<>():<lambda()>().echo<>():<lambda()>()' is not a constant expression
      6 |     [&]{ std::print("{} ", args...); }();
        |         ~~~~~^~~~~
<source>:6:38: error: 'echo<>():<lambda()>' called in a constant expression
<source>:6:5: note: 'echo<>():<lambda()>' is not usable as a 'constexpr' function because:
      6 |     [&]{ std::print("{} ", args...); }();
        |         ^~~~~
<source>:6:20: error: call to non-'constexpr' function 'void std::print(format_string<Args ...>, _Args&&... _args) [with _Args = {}; format_string<Args ...> = basic_format_string<char>]'
      6 |     [&]{ std::print("{} ", args...); }();
        |         ~~~~~^~~~~
In file included from <source>:1:
/opt/compiler-explorer/gcc-trunk-20250106/include/c++/15.0.0/print:117:5: note: 'void
      std::print(format_string<Args ...>, _Args&&... _args) [with _Args = {}; format_string<Args ...> =
      basic_format_string<char>]' declared here
      117 |     print(format_string<Args...> __fmt, _Args&&... _args)
          |     ~~~~~^~~~~
<source>:6:21: note: 'echo<>():<lambda()>' was promoted to an immediate function because its body contains
      an immediate-escalating expression 'std::basic_format_string<char>("{}")'
      6 |     [&]{ std::print("{} ", args...); }();
        |         ^~~~~

```

The solution here is the same: we ensure that the only kinds of expressions that are immediate-escalating are those that are not `constexpr`-erroneous.

§ 4.6.2 Exceptions

The next question becomes exactly what kinds of expressions should produce constexpr-erroneous values. Obviously `std::constexpr_print_str`, that's the point of the paper. I think in the future we'll want to tackle things like `std::abort()`, `std::terminate()`, possibly even any `[[noreturn]]` function.

But what about escaped exceptions? Consider this example:

```
constexpr std::array<int, 3> data = {1, 2, 3};

int v1 = data.at(5);

void f() {
    try {
        static int v2 = data.at(6);
    } catch (...) {

    }
}

int main() {
    f();
}
```

Here, we have two variables with static storage duration whose constant initialization fails due to an uncaught exception. `v1` gets elevated into a runtime failure. `v2`, though, isn't any kind of failure at all — we catch the exception at compile time. We have a choice to make: do we consider uncaught exceptions to be erroneous or not? Doing so would catch the initialization failure of `v1` at compile time, but it would *also* mean that the initialization of `v2` becomes a compile time error as well.

While it's certainly possible to have static initialization fail with a caught exception at runtime, I think it's exceedingly unlikely to have a meaningful case of static initialization failure *that could be constant* fail in this way. It'd be one thing if instead of the `6` above, the index was a (non-constant) parameter of `f`, where potentially one call to `f` could fail but the next might succeed. If the expression is otherwise constant, *every* call will fail. That seems like strange code to me.

The problem is, the idea I have for constexpr-erroneous is that a constexpr-erroneous expression isn't recoverable. But an exception, by definition, is:

```
constexpr void throw_up() {
    // some constexpr-only type
    throw std::meta::exception(...);
}

template <class F>
constexpr bool attempt_to(F f) {
    try {
        f();
        return true;
    } catch (...) {
        return false;
    }
}

static_assert(attempt_to([]{ throw_up(); }));
```

Currently, `throw_up()` is immediate-escalating, causing the appropriate specialization of `attempt_to` to become `constexpr`. And at that point, that specialization is actually a constant expression (that returns `true`). If we labelled `throw_up()` as erroneous (because of the uncaught exception), `attempt_to()` wouldn't be able to recover. But since it can, I don't think we can label escaped exceptions as constexpr-erroneous.

§ 5 Proposal

This paper proposes the following:

1. Introduce a new compile-time diagnostic API that only has effect if manifestly constant evaluated: `std::constexpr_print_str([tag], msg)`.
2. Introduce a new compile-time error APIs, that only has effect if manifestly constant evaluated: `std::constexpr_error_str(tag, msg)` will both cause the program to be ill-formed additionally cause the expression to not be a constant expression, emitting the message under the provided tag (which can be used in an implementation-defined way to control whether the diagnostic is emitted). EWG took a poll in February 2023 to encourage work on the ability to print multiple errors per constant evaluation but still result in a failed TU:

SF	F	N	A	SA
5	10	3	1	0

However, this design choice seems unmotivated and would require two differently-named error functions - first taking `string_view` now and then the full format API later. There is some precedent to this (e.g. Catch2 has `CHECK` and `REQUIRE` macros - the first of which cause a test to

fail but continue running to print further diagnostics, while the second causes the test to fail and immediately halt execution), but in a constant evaluation context with the freedom to form arbitrary messages, I don't think this distinction is especially useful. The REQUIRE functionality is critical, the CHECK one less so.

3. Introduce a new compile time warning API that only has effect if manifestly constant evaluated: `std::constexpr_warning_str(tag, msg)`. This will emit a warning containing the provided message under the provided tag, which can be used in an implementation-defined way to control whether the diagnostic is emitted.
4. Pursue `constexpr std::format(fmt_str, args...)`, which would then allow us to extend the above API with `std::format`-friendly alternatives.
5. Introduce the concept of constexpr-erroneous expressions to help word this.

§ 5.1 Implementation Experience

Hana Dusíková has a partial implementation of this paper in clang. Specifically, `std::constexpr_print_str` and a simplified version of `std::constexpr_error_str` that does not include a tag. This program fails compilation as desired:

```
constexpr int foo(int a) {
    if (a == 0) {
        std::constexpr_error_str("can't call with a == 0");
    }

    return a;
}

int a = foo(2); // OK
int b = foo(0); // error: custom constexpr error: 'can't call with a == 0'
```

§ 6 Wording

We don't quite have `constexpr std::format` yet (although with the addition of [P2738R1] we're probably nearly the whole way there), so the wording here only includes (1) and (2) above - with the understanding that a separate paper will materialize to produce a `constexpr std::format` and then another separate paper will add `std::constexpr_print` and `std::constexpr_error` (the nicer names, with the more user-friendly semantics).

Alter how static initialization works to ensure there's no fallback to runtime initialization in some cases, in 6.9.3.2 [basic.start.static]:

- 2 *Constant initialization* is performed if a variable with static or thread storage duration is constant-initialized ([`expr.const`]). *If the full-expression of the initialization of the variable is a constexpr-erroneous expression ([`expr.const`]), the program is ill-formed. Otherwise, if $\nexists$ constant initialization is not performed, a variable with static storage duration ([`basic.stc.static`]) or thread storage duration ([`basic.stc.thread`]) is zero-initialized ([`dcl.init`]).*

Introduce the notion of constexpr-erroneous in 7.7 [`expr.const`]:

- x An expression *E* has *constexpr-erroneous value* it evaluates an invocation of `std::constexpr_error_str` ([`meta.const.eval`]). It is implementation-defined whether an invocation of `std::constexpr_warning_str` has a constexpr-erroneous value. [Drafting note: I suspect that we will eventually make more things produce constexpr-erroneous values, like maybe calls to `std::terminate()`, `std::unreachable()`, maybe any [[noreturn]] function, etc.]
- 10 An expression *E* is a *core constant expression* unless the evaluation of *E* following the rules of the abstract machine ([`intro.execution`]), would evaluate one of the following:
 - (10.1) — [...]
 - [...]
 - (10.32) — an expression with constexpr-erroneous value.
- y An expression *E* is a *constexpr-erroneous expression* if evaluation of *E* is not a core constant expression due to evaluation of an expression with constexpr-erroneous value.

[Example 1:

```
constexpr int foo(int a) {
    if (a == 0) {
        std::constexpr_error_str("reject-zero", "can't call with a == 0");
    }

    return a;
}
```

]

```

int x = foo(2); // OK, constant-initialized
int y = foo(0); // error: the initialization of y is a constexpr-erroneous expression
—end example ]

```

Make constexpr-erroneous immediate expressions hard errors, so they don't escalate:

- 25 An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and it is either
- (25.1) — a potentially-evaluated *id-expression* that denotes an immediate function that is not a subexpression of an immediate invocation, or
- (25.2) — an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.
- z An immediate-escalating expression shall not be constexpr-erroneous.
- 26 An *immediate-escalating* function is:
- (26.1) — the call operator of a lambda that is not declared with the consteval specifier,
- (26.2) — a defaulted special member function that is not declared with the consteval specifier, or
- (26.3) — a function that results from the instantiation of a templated entity defined with the constexpr specifier.
- An immediate-escalating expression shall appear only in an immediate-escalating function.

Add to 21.3.3 [meta.type.synop]:

```

// all freestanding
namespace std {
    // ...

    // [meta.const.eval], constant evaluation context
    constexpr bool is_constant_evaluated() noexcept;
    consteval bool is_within_lifetime(const auto*) noexcept;

    + // [meta.const.msg], emitting messages at compile time
    + struct tag-string; // exposition-only
    +
    + constexpr void constexpr_print_str(string_view) noexcept;
    + constexpr void constexpr_print_str(tag-string, string_view) noexcept;
    + constexpr void constexpr_warning_str(tag-string, string_view) noexcept;
    + constexpr void constexpr_error_str(tag-string, string_view) noexcept;

}

```

Add a new clause after 21.3.11 [meta.const.eval] named “Emitting messages at compile time”:

- 1 The facilities in this subclause are used to emit messages at compile time.
- 2 A call to any of the functions defined in this subclause may produce a diagnostic message during constant evaluation. The text from a `string_view`, *M*, is formed by the sequence of *M.size()* code units, starting at *M.data()*, of the ordinary literal encoding ([lex.charset]).
- ```

 struct tag-string { // exposition-only
 private:
 string_view str; // exposition-only

 public:
 template<class T> consteval tag-string(const T& s);
 };

 template<class T> consteval tag-string(const T& s);

```
- 3 **Constraints:** `const T&` models `convertible_to<string_view>`.
- 4 **Effects:** Direct-non-list-initializes *str* with *s*.
- 5 **Remarks:** A call to this function is not a core constant expression unless every character in *str* is either a *nondigit*, a *digit*, or a *-*.
- ```

    constexpr void constexpr_print_str(string_view msg) noexcept;
    constexpr void constexpr_print_str(tag-string tag, string_view msg) noexcept;

```
- 6 **Effects:** During constant evaluation, a diagnostic message is issued including the text of *msg*. Otherwise, no effect.
- 7 **Recommended practice:** Implementations should include the text of *tag.str*, if provided, in the diagnostic.

```
constexpr void constexpr_warning_str(tag-string tag, string_view msg) noexcept;
```

8 *Effects:* During constant evaluation, a diagnostic message is issued including the text of msg. Otherwise, no effect.

9 *Recommended practice:* Implementations should issue a warning in such cases and provide a mechanism allowing users to either opt in or opt out of such warnings based on the value of tag.str.

```
constexpr void constexpr_error_str(tag-string tag, string_view msg) noexcept;
```

10 *Effects:* During constant evaluation, a diagnostic message is issued including the text of msg [*Note 1:* evaluation of such a call is constexpr-erroneous ([expr.const]) — *end note*]. Otherwise, no effect.

11 *Recommended practice:* Implementations should include the text of tag.str in the diagnostic.

§ 7 References

[N4433] Michael Price. 2015-04-09. Flexible static_assert messages.

<https://wg21.link/n4433>

[P0596R1] Daveed Vandevoorde. 2019-10-08. Side-effects in constant evaluation: Output and consteval variables.

<https://wg21.link/p0596r1>

[P2291R3] Daniil Goncharov, Karaev Alexander. 2021-09-23. Add Cxxpr Modifiers to Functions to_chars and from_chars for Integral Types in Header.

<https://wg21.link/p2291r3>

[P2564R3] Barry Revzin. 2022-11-11. consteval needs to propagate up.

<https://wg21.link/p2564r3>

[P2738R1] Corentin Jabot, David Ledger. 2023-02-13. constexpr cast from void*: towards constexpr type-erasure.

<https://wg21.link/p2738r1>

[P2741R0] Corentin Jabot. 2022-12-09. user-generated static_assert messages.

<https://wg21.link/p2741r0>

[P2741R3] Corentin Jabot. 2023-06-16. user-generated static_assert messages.

<https://wg21.link/p2741r3>

[P2758R0] Barry Revzin. 2023-01-13. Emitting messages at compile time.

<https://wg21.link/p2758r0>

[P2758R1] Barry Revzin. 2023-12-09. Emitting messages at compile time.

<https://wg21.link/p2758r1>

[P2758R2] Barry Revzin. 2024-02-15. Emitting messages at compile time.

<https://wg21.link/p2758r2>

[P2758R3] Barry Revzin. 2024-05-19. Emitting messages at compile time.

<https://wg21.link/p2758r3>

1. A previous revision of the paper [explained why](#): `static_assert(cond, "T{} must be valid expression")` is a valid assertion today. Adopting the format API would break this assertion - were it to fire. However, given that this is a static assertion, perhaps there's room to maneuver here.↵